UVM

LAUREATE INTERNATIONAL UNIVERSITIES®

ACTIVIDAD V

Profesor

Cuauhtémoc León Puertos

Alumnos

Cristian Daniel López Rios

Fecha de Entrega

07-Agosto-2023



ACTIVIDAD V:

PROPUESTA DE EJERCICIO CON FUNCIONES, ESTRUCTURAS Y ARREGLOS

Fecha:07/08/2023

Nombre del estudiante: Cristian Daniel López Rios

Nombre del docente: Cuahtémoc León Puertos

Resuelve los ejercicios aplicando los conocimientos sobre:

- *Lógica de programación*
- *Funciones*
- *Estructuras*
- *Arreglos*

Ejercicio 1. Procedimientos y funciones

A partir de los recursos revisados acerca de procedimientos y funciones, resuelve lo siguiente:

Realiza un programa que determine si un número es par o impar, se deben de implementar las siguientes funciones:

a. La función que evalúa si un número es par o impar.
b. La función *main* que hace el uso correcto de la función definida anteriormente.



Ejemplo de entrada
Ingrese el número a evaluar **8**
Ejemplo se salida
El número 8 es par

Código

```
#include <stdio.h>
//#include <unist.h>
#include <windows.h>

int main(){
```

```
    //entrada de valores
    int input, rem;
    int calc_module(int a);
    int calc_module_b(int a, int b);

    printf("Ingrese el número a evaluar:\t");
    scanf("%d", &input);
    rem = calc_module(input);

    if(rem != 0){
           printf("El número %d es impar.\n", input);
    }else{
           printf("El número %d es par.\n", input);
    }
    sleep(1000);
 return 0;
}

int calc_module(int a){
    calc_module_b(a, 2);

}

int calc_module_b(int z, int x){
    return z % x;
}
```

Ejercicio 2. Arreglos de una dimensión

A partir de los recursos revisados acerca arreglos de una dimensión y funciones, resuelve lo siguiente:

Realiza un programa que llene un arreglo unidimensional (vector) de tamaño n de calificaciones y calcule el promedio, usar funciones.
Las funciones que debe de implementar son las siguientes:

a. Función para llenar el arreglo
b. Función para calcular el promedio
c. La función *main,* en donde se deben de llamar a las demás funciones de la forma correcta.



$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

L	M	M	J	V	S	D
6	3	4	7	5	4	8

$$\bar{x} = \frac{6+3+4+7+5+4+8}{7} = 5.28$$

Ejemplo de entrada
¿Cuantas calificaciones desea ingresar? **7**
Ingresa la calificación 1: **6**
Ingresa la calificación 2: **3**
Ingresa la calificación 3: **4**
Ingresa la calificación 4: **7**
Ingresa la calificación 5: **5**
Ingresa la calificación 6: **4**
Ingresa la calificación 7: **8**

Ejemplo de salida
El promedio es: **5.28**

Código

```
#include <stdio.h>
#include <stdlib.h>

void inputElements(int **array, int *size) {
    printf("Enter the size of the array: ");
    scanf("%d", size);

    *array = (int *)malloc(*size * sizeof(int));

    printf("Enter %d elements:\n", *size);
    for (int i = 0; i < *size; i++) {
        scanf("%d", &(*array)[i]);
    }
}

double calculateAverage(int *array, int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += array[i];
    }
    return (double)sum / size;
}

int main() {
    int *array = NULL;
    int size;
    double average;
    double calculateAverage(int *array, int size)
    void inputElements(int **array, int *size)
```

```
    inputElements(&array, &size);
    average = calculateAverage(array, size);

    printf("Average of array elements: %.2f\n", average);

    free(array);

    return 0;
}
```

Para el archivo en C:

- Primero se compila el archivo con terminación *.c con el comando gcc.
- Posteriormente se llama al archivo a.out que es resultado de compilar *.c.

```
¡Cuantas calificaciones desea ingresar? 4
Ingresa la calificaci¾n 1: 5
Ingresa la calificaci¾n 2: 5
Ingresa la calificaci¾n 3: 10
Ingresa la calificaci¾n 4: 10

El promedio es: 7.50
```

Ejercicio 3. Arreglos bidimensionales

A partir de los recursos revisados acerca arreglos bidimensionales y funciones, resuelve lo siguiente:

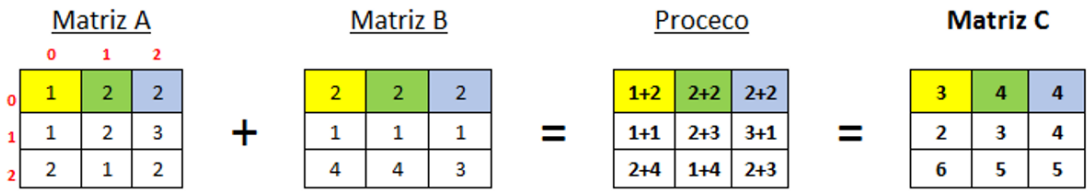
Realiza un programa que calcule la suma de matrices (arreglos bidimensionales), de dimensión mxn, use funciones.

Las funciones que debe de implementar son las siguientes:

a. Función para llenar las matrices a sumar
b. Función que calcule la suma de matrices
c. Función que muestre el resultado de la suma de matrices.
d. Función *main* que llama de la forma de la forma correcta a las demás funciones.



SUMA DE MATRICES



Ejemplo de entrada:

Ingrese el número de filas: 3
Ingrese el número de columnas: 3

Matriz A
Ingrese el valor de a_{11}: 1
Ingrese el valor de a_{12}: 2
Ingrese el valor de a_{13}: 2
Ingrese el valor de a_{21}: 1
Ingrese el valor de a_{22}: 2
Ingrese el valor de a_{23}: 3
Ingrese el valor de a_{31}: 2
Ingrese el valor de a_{32}: 1
Ingrese el valor de a_{33}: 2

Matriz B
Ingrese el valor de b_{11}: 2
Ingrese el valor de b_{12}: 2
Ingrese el valor de b_{13}: 2
Ingrese el valor de b_{21}: 1
Ingrese el valor de b_{22}: 1
Ingrese el valor de b_{23}: 1
Ingrese el valor de b_{31}: 4
Ingrese el valor de b_{32}: 4
Ingrese el valor de b_{33}: 3

Ejemplo de salida

Matriz C = Matriz A+ Matriz B
C_{11} = 3
C_{12} = 4
C_{13} = 4

$C_{21} = 2$
$C_{22} = 3$
$C_{23} = 4$
$C_{31} = 6$
$C_{32} = 5$
$C_{33} = 5$

Código

```
#include <stdio.h>
#include <stdlib.h>

// Función para llenar la matriz.
void fillMatrix(int matrix[][10], int rows, int cols, char
matrixName) {
     int i,j;
    printf("Matriz %c\n", matrixName);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            printf("Ingrese el valor de %c%d%d: ", matrixName,
i + 1, j + 1);
            scanf("%d", &matrix[i][j]);
        }
    }
}

// Función para agregar dos matrices.
void addMatrices(int matrixA[][10], int matrixB[][10], int
matrixC[][10], int rows, int cols) {
     int i, j;
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            matrixC[i][j] = matrixA[i][j] + matrixB[i][j];
        }
    }
}

// Función para mostrar la matriz.
void displayMatrix(int matrix[][10], int rows, int cols, char
matrixName) {
     int i,j;
    printf("\nMatriz %c\n", matrixName);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            printf("%c%d%d = %d\n", matrixName, i + 1, j + 1,
matrix[i][j]);
        }
```

```
    }
}

int main() {
    fillMatrix(int matrix[][10], int rows, int cols, char
matrixName);
    displayMatrix(int matrix[][10], int rows, int cols, char
matrixName);
    addMatrices(int matrixA[][10], int matrixB[][10], int
matrixC[][10], int rows, int cols);
    int rows, cols;
    printf("Ingrese el número de filas: ");
    scanf("%d", &rows);
    printf("Ingrese el número de columnas: ");
    scanf("%d", &cols);

    int matrixA[10][10], matrixB[10][10], matrixC[10][10];

    fillMatrix(matrixA, rows, cols, 'A');
    fillMatrix(matrixB, rows, cols, 'B');

    addMatrices(matrixA, matrixB, matrixC, rows, cols);

    displayMatrix(matrixC, rows, cols, 'C');
    sleep(1000);

    return 0;
}
```

Para el archivo en C:

- Primero se compila el archivo con terminación *.c con el comando gcc.
- Posteriormente se llama al archivo a.out que es resultado de compilar *.c.



```
Ingrese el valor de A31: 1
Ingrese el valor de A32: 2
Ingrese el valor de A33: 3
Matriz B
Ingrese el valor de B11: 1
Ingrese el valor de B12: 2
Ingrese el valor de B13: 3
Ingrese el valor de B21: 4
Ingrese el valor de B22: 5
Ingrese el valor de B23: 6
Ingrese el valor de B31: 4
Ingrese el valor de B32: 5
Ingrese el valor de B33: 6

Matriz C
C11 = 2
C12 = 4
C13 = 6
C21 = 5
C22 = 7
C23 = 9
C31 = 5
C32 = 7
C33 = 9
```

Ejercicio 4. Estructuras

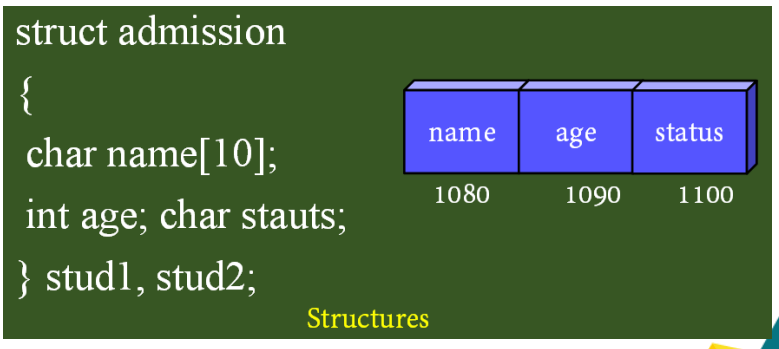
A partir de los recursos revisados acerca de los tipos de datos *struct,* resuelve lo siguiente:

Realizar un programa que permita guardar en un tipo *struct* los datos concernientes a un estudiante, los datos de interés son:

- Matricula
- Nombre
- Carrera
- Promedio
- Dirección

Se debe de realizar las siguientes funciones:

a. Captura de los datos
b. Mostrar los datos
c. La función main que llame de la forma correcta a las funciones captura y mostrar.



Ejemplo de entrada

Ingrese la matrícula del estudiante: **12345**

Ingrese el nombre del estudiante: **Juan Pérez**

Ingrese la carrera del estudiante: **Ingeniería en Sistemas**

Ingrese el promedio del estudiante: **9.5**

Ingrese la dirección del estudiante: **Calle Juárez #16, Colonia Centro, Macuspana Tabasco**

Ejemplo de salida

La matrícula del estudiante es: **12345**

El nombre del estudiante es: **Juan Pérez**

La carrera del estudiante es: **Ingeniería en Sistemas**

El promedio del estudiante es: **9.5**

La dirección del estudiante es: **Calle Juárez #16, Colonia Centro, Macuspana Tabasco**

Código

```
#include <stdio.h>
#include <stdlib.h>

// Definición de la estructura para el estudiante
struct Estudiante {
    int matricula;
    char nombre[50];
    char carrera[50];
    float promedio;
    char direccion[100];
};

// Función para capturar los datos del estudiante
void capturarDatos(struct Estudiante *estudiante) {
    printf("Ingrese la matrícula: ");
    scanf("%d", &(estudiante->matricula));

    printf("Ingrese el nombre: ");
    scanf(" %[^\n]", estudiante->nombre);

    printf("Ingrese la carrera: ");
    scanf(" %[^\n]", estudiante->carrera);

    printf("Ingrese el promedio: ");
    scanf("%f", &(estudiante->promedio));
```

```
    printf("Ingrese la dirección: ");
    scanf(" %[^\n]", estudiante->direccion);
}

// Función para mostrar los datos del estudiante
void mostrarDatos(struct Estudiante estudiante) {
    printf("\nDatos del estudiante:\n");
    printf("Matrícula: %d\n", estudiante.matricula);
    printf("Nombre: %s\n", estudiante.nombre);
    printf("Carrera: %s\n", estudiante.carrera);
    printf("Promedio: %.2f\n", estudiante.promedio);
    printf("Dirección: %s\n", estudiante.direccion);
}

int main() {
    void mostrarDatos(struct Estudiante estudiante);
    void capturarDatos(struct Estudiante *estudiante);
    struct Estudiante estudiante;

    capturarDatos(&estudiante);
    mostrarDatos(estudiante);

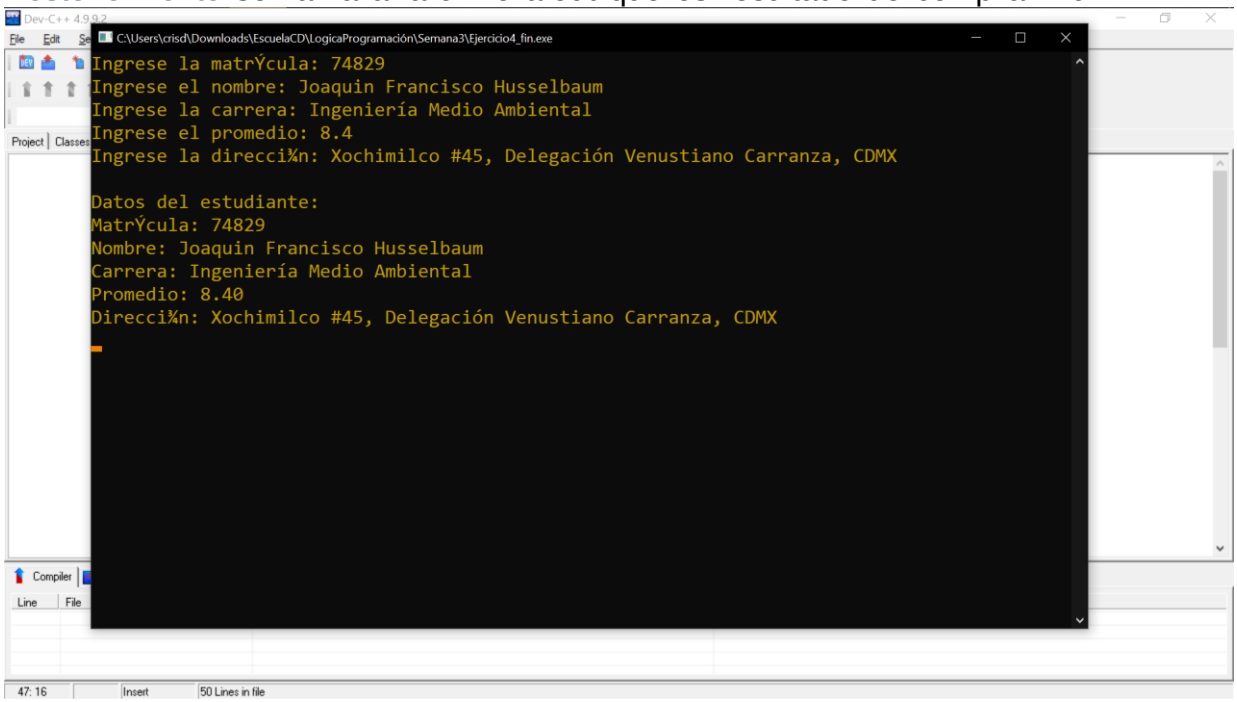
    sleep(10000);
    return 0;
}
```

Para el archivo en C:

- Primero se compila el archivo con terminación *.c con el comando gcc.



- Posteriormente se llama al archivo a.out que es resultado de compilar *.c.



Ejercicio 5. Cuestionario

1. Revisa nuevamente el documento "Funciones en C" disponible también en los materiales básicos de la unidad y resuelve el cuestionario de evaluación que aparece en la página 98.

1. ¿Qué es una función?

Una función es una reserva de memoria en la RAM del dispositivo que permite declarar variables y estructuras de control para poder hacer operaciones de forma modular.

2. ¿Qué es la llamada a una función?

Llamar a una función es hacer una declaración con el nombre de la función y los argumentos necesarios para llevar a cabo la operación.

3. ¿Cuál es la diferencia entre argumentos formales y argumentos reales?

Como lo menciona Márquez, et Al. p. 212 "Los parámetros formales (también denominados parámetros ficticios) … Por otro lado, los valores con que se invoca

a la función se denominan argumentos o parámetros reales las variables que se declaran dentro del paréntesis junto con la función.”

4. ¿La instrucción *return* sólo puede aparecer una vez dentro de una función? Justifica brevemente

Se puede utilizar la función return más de una vez dentro de una estructura de control, de lo contrario no es necesario regresar más de un valor de una función.

5. ¿Podemos llamar a una función más de una vez desde distintas partes de un programa? Justifica brevemente

Sí se puede llamar a una función más de una vez para realizar la operación requerida, para ello se debe declarar la función y pasar los argumentos necesarios para que lleve a cabo la operación.

```
int sum(int a, int b){
c = a + b;
return c;
}

int main(){
int x, y;
x = sum(45, 44);
y = sum(23, 10);
printf(“%d %d”, x, y);
Return 0;
}
```

Como se observa, en este ejemplo, la función se llama dos veces.

6. ¿Qué diferencia existe entre el paso de argumentos por valor y el paso de argumentos por referencia?

Los argumentos pasados por valor es la forma convencional de declarar una función con argumentos inmutables. Por otro lado Márquez, et Al. p.216 nos describe que los argumentos por referencia son aquellos que hacen referencia a la dirección de hospedaje de una variable en la memoria interna del servidor.

7. ¿De qué se vale C para implementar el paso de argumentos por referencia?

Márquez, et Al. p.216-217 se utilizan los operadores & y *. El operador & se antepone a una variable, dando con ello acceso a su dirección. El operador * es un apuntador que “apunta” a la dirección de la variable pasada como argumento.

8. ¿Cuándo es recursiva una función?

Una función es recursiva cuando hace referencia a sí mismo con los argumentos ya procesados dentro de la función.

9. ¿Qué son las funciones de biblioteca?

Son funciones que el preprocesador obtiene de un encabezado que generalmente está localizado en la carpeta de una biblioteca.

10. ¿Cómo se pasan los argumentos a la función *scanf*?

Para pasar argumentos a la función scanf(); se utiliza el operador concatenador & que permite dirigir datos a la dirección en memoria de una variable.

3. Redacta una **conclusión** en la que expliques la importancia de la definición de funciones para el uso de arreglos y estructuras de datos en el entorno de la programación estructurada.

La técnica de programación requiere del reciclado de funciones que diversas personas han contribuido a escribir. Por esta razón, los programas escritos en código deben ser modulares y flexibles. Esto permite que se desarrollen programas complejos con mayor velocidad y facilidad. De esta forma, se cortan los tiempos de producción y se facilita el mantenimiento a los programas informáticos.

Lenguajes como C, son lenguajes muy utilizados en compiladores y sistemas operativos, por esta razón tiene ciertas restricciones al localizar memoria y reglas que impiden que los arreglos se modifiquen como variables dinámicas. Por esta razón, se deben crear funciones “modularizen” el hospedaje de memoria y modifiquen el tamaño de un arreglo o lista de valores.

A partir de la creación de arreglos de diferentes dimensiones se puede extender el uso de funciones enfocadas en el hospedaje de memoria que permitan crear espacio para variables de diferentes tipos e incluso modificar la forma de acceso a estas variables. De esta manera se pueden crear diferentes estructuras de datos que guarden diferentes tipos de datos o incluso que estén optimizadas para ser accedidas y/o que devuelvan valores de cierta forma como filas o nodos.

Referencias:

Márquez, G.; Osorio, S.; Olvera, N. (2011). Introducción a la Programación Estructurada en C. México, Pearson Educación de México.